Branch:Computer Science &Engg.

NOTES

Unit 1 Basic Structure of Computer

Hardware vs. Software

Software describes a **collection of programs and procedures** that perform tasks on a computer. Software is an ordered sequence of instructions that change the state of a computer's hardware.There are three general types of software:

- System software
- Programming software
- Application software

When you think of computer science, software is probably what comes to mind. Software is what developers actually **code**. Those programs are then installed onto a hard drive.

Hardware is anything **physically connected to a computer**. For example, your display monitor, printer, mouse, and hard drive are all hardware components.

Hardware and software **interact with each other**. The software "tells" the hardware which tasks to perform, and hardware makes it possible to actually perform them.

Hardware	Software
Physical devices that store and run software	Collection of coded instructions that all us to interact with a computer
Works as the delivery system	Performs specific tasks

Hardware	Software
Monitor, printer, scanners , label makers, routers and hard drive	Adobe, Google Chrome, Microsoft Excel, Spotify
Hardware begins functioning when software is loaded.	Software must be installed on hardware
Hardware will wear down over time	Software will not wear down, but it is vulnerable to bugs / becoming outdated

Hardware components

Now that we understand the difference between hardware and software, let's learn about the hardware components of a computer system. Remember: hardware includes the physical parts of a computer that the software instructs.

CPU

The Central Processing Unit (CPU) is a physical object that **processes information** on a computer. It takes data from the main memory, processes it, and returns the modified data into the main memory. It is comprised of two sub-units:

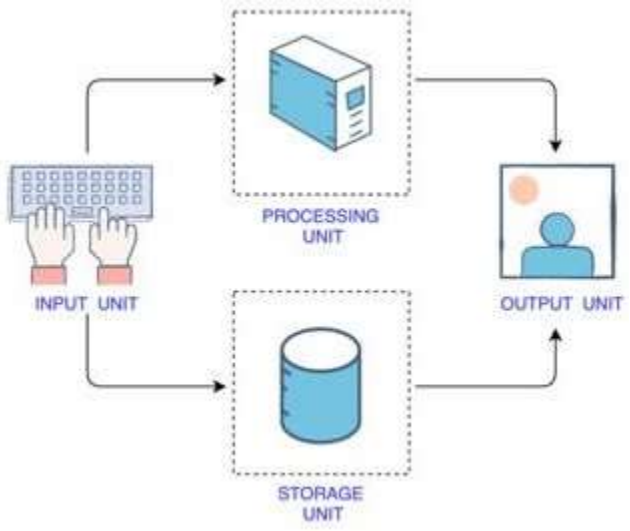
- **The control unit (CU):** controls data flow from and into the main memory
- **The arithmetic and logic unit (ALU):** processes the data

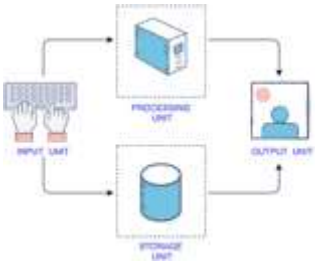
Von Neumann architecture

This computer architecture design, created by John von Neumann in 1945, is still used in most computers produced today. The Von Neumann architecture is based on the concept of a **stored-program computer**. Instruction and program data are stored in the same memory.

This architecture includes the following components:

- Control Unit
- Inputs/Outputs
- Arithmetic and Logic Unit (ALU)
- Memory Unit
- Registers





The Von Neumann Architecture.

Input and output units

The **input unit** takes inputs from the real world or an input device and converts that data into streams of bytes. Common input devices include a keyboard, mouse, microphone, camera, and USB.

The **output unit**, on the other hand, takes the processed data from the storage of CPU and represents it in a way a human can understand. Common output devices include a monitor screens, printers, and headphones.

Storage Units

After the data is retrieved and converted, it must be stored in the **memory**. The storage unit or memory is the physical memory space. It is divided into byte-sized storage locations.

Storage Unit example

A storage contains millions of bytes of memory to store anything we want on our computer. To store a bit of data in computer memory, we use a **circuit**, called a latch, that stores the previous input unless it's reset. We can create a circuit using a:

- S-R latch
- Gated S-R latch
- D latch

Memory

There are components to a computer's hardware memory. **Main memory** or **random access memory (RAM)** is the physical memory space inside a computer. It stores data and instructions that can directly be accessed by the CPU. Computers usually have a *limited amount* of main memory to store all your data.

That is when **secondary storage** comes into use. Secondary storage augments the main memory and holds data and programs that are not needed immediately.

Secondary storage devices include hard drives, compact discs (CD), USB flash drives, etc. Secondary storage devices cannot be directly accessed by the CPU.

Software components

Now let's discuss the different software components that we need to have a functioning computer. Remember: software comprises the set of programs, procedures, and routines associated needed to operate a computer.

Machine language

A computer can only process binary: a stream of ones and zeros. Binary is the computer's language. Instructions for the computer are also stored as ones and zeros that the computer must decode and execute.

Assembly language

Assembly language is a human-readable instruction mode that translates binary opcode into assembly instruction. A CPU cannot process or execute assembly instructions, so an encoder is required that can convert assembly language to machine language.

Assembler

An assembler translates an assembly language program into machine language.

High-level languages

Assembly language is referred to as a **low-level language** because it's a lot like machine language. To overcome these shortcomings, high-level languages were created.

These are called **programming languages**, and they allow us to create powerful, complex, human-readable programs without large numbers of low-level instructions. Some of the most famous high-level languages are:

- Python
- C++

- JAVA

  Print “Hello World”

Run

Example of "Hello World" in Python

Addressing Modes

Addressing Modes– The term addressing modes refers to the way in which the operand of an instruction is specified. The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually executed.

Addressing modes for 8086 instructions are divided into two categories:
1) Addressing modes for data

2) Addressing modes for branch

The 8086 memory addressing modes provide flexible access to memory, allowing you to easily access variables, arrays, records, pointers, and other complex data types. The key to good assembly language programming is the proper use of memory addressing modes.

An assembly language program instruction consists of two parts

Opcode	Operand

The memory address of an operand consists of two components:

According to different ways of specifying an operand by 8086 microprocessor, different addressing modes are used by 8086.

Addressing modes used by 8086 microprocessor are discussed below:

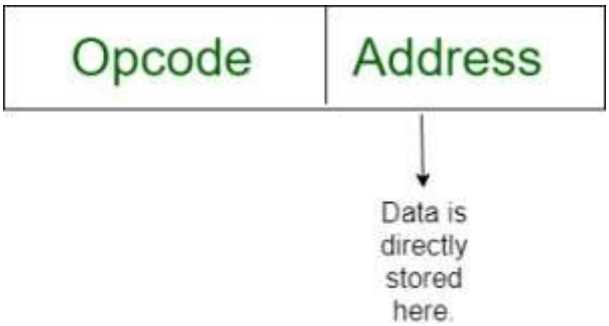
- **Implied mode:**: In implied addressing the operand is specified in the instruction itself. In this mode the data is 8 bits or 16 bits long and data is the part of instruction.Zero address instruction are designed with implied addressing mode. **Instruction**

Data

  Example: CLC (used to reset Carry flag to 0)

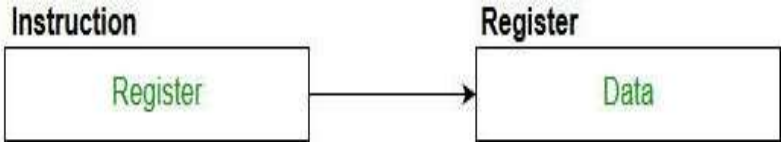
- **Immediate addressing mode (symbol #):**In this mode data is present in address field of instruction .Designed like one address instruction format. **Note:**Limitation in the immediate mode is that the range of constants are restricted by size of

address field.



Example: MOV AL, 35H (move the data 35H into AL register)

- **Register mode:** In register addressing the operand is placed in one of 8 bit or 16 bit general purpose registers. The data is in the register that is specified by the instruction. *Here one register reference is required to access the data.*



Example: MOV AX,CX (move the contents of CX register to AX register)

- **Register Indirect mode**: In this addressing the operand's offset is placed in any one of the registers BX,BP,SI,DI as specified in the instruction. The effective address of the data is in the base register or an index register that is specified by the instruction. *Here two register reference is required to access the data.*

The 8086 CPUs let you access memory indirectly through a register using the register indirect addressing modes.

- MOV AX, [BX](move the contents of memory location s

  addressed by the register BX to the register AX)

- **Auto Indexed (increment mode)**: Effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next consecutive memory location.**(R1)+**. *Here one register reference,one memory reference and one ALU operation is required to access the data.*

  Example:

- Add R1, (R2)+ // OR

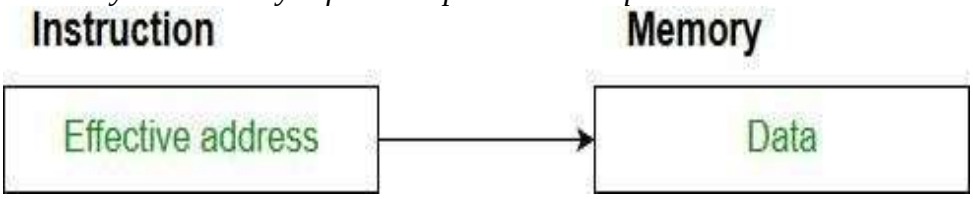
- R1 = R1 +M[R2]

  R2 = R2 + d

  Useful for stepping through arrays in a loop. R2 – start of array d – size of an element

- **Auto indexed (decrement mode)**: Effective address of the operand is the contents of a register specified in the instruction. Before accessing the operand, the contents of this register are automatically decremented to point to the previous consecutive memory location. –**(R1)**
  Here one register reference,one memory reference and one ALU operation is required to access the data.
  Example:
  Add R1,-(R2) //OR

  R2 = R2-*d*
  R1 = R1 + M[R2]
  Auto decrement mode is same as auto increment mode. Both can also be used to implement a stack as push and pop . Auto increment and Auto decrement modes are useful for implementing "Last-In-First-Out" data structures.
- **Direct addressing/ Absolute addressing Mode (symbol [ ]):** The operand's offset is given in the instruction as an 8 bit or 16 bit displacement element. In this addressing mode the 16 bit effective address of the data is the part of the instruction. *Here only one memory reference operation is required to access the data.*



  Example:ADD AL,[0301] //add the contents of offset address 0301 to AL

- **Indirect addressing Mode (symbol @ or ())**:In this mode address field of instruction contains the address of effective address.Here two references are required. 1st reference to get effective address. 2nd reference to access the data.
  Based on the availability of Effective address, Indirect mode is of two kind:

  1. Register Indirect:In this mode effective address is in the register, and corresponding register name will be maintained in the address field of an instruction. *Here one register reference,one memory reference is required to access the data.*
  2. Memory Indirect:In this mode effective address is in the memory, and corresponding memory address will be maintained in the address field of an instruction. *Here two memory reference is required to access the data.*
- **Indexed addressing mode**: The operand's offset is the sum of the content of an index register SI or DI and an 8 bit or 16 bit displacement.
  Example:MOV AX, [SI +05]

- **Based Indexed Addressing:** The operand's offset is sum of the content of a base register BX or BP and an index register SI or DI.
  Example: ADD AX, [BX+SI]

  Based on Transfer of control, addressing modes are:
  - **PC relative addressing mode:** PC relative addressing mode is used to implement intra segment transfer of control, In this mode effective address is obtained by adding displacement to PC.

- EA= PC + Address field value

  PC= PC + Relative value.

- **Base register addressing mode:**Base register addressing mode is used to implement inter segment transfer of control.In this mode effective address is obtained by adding base register value to address field value.
- EA= Base register + Address field value.
- PC= Base register + Relative value.

Advantages of Addressing Modes

1. To give programmers to facilities such as Pointers, counters for loop controls, indexing of data and program relocation.
2. To reduce the number bits in the addressing field of the Instruction.

Sequence Programming

Before diving into a detailed explanation of a sequence in programming, let's take a look at a real world example.

Let's take the example of organizing a wardrobe of clothes. Typically, the wardrobe will be messy and cluttered. The task that is assigned is to declutter the wardrobe and arrange the clothes. How to complete this assigned task?

Firstly, one may choose to tackle the wardrobe shelf by shelf, or may empty the whole wardrobe and then sort all the clothes into piles. One may also sort the clothes according to the season or according to its colors. But no matter which way they choose to accomplish the task, the task itself can be ordered in a sequence. The sequence may look something like this:

1. Empty the wardrobe.
2. Sort clothes into colors/functionality/season.
3. Fold the clothes.
4. Arrange it into piles.
5. Put the piles back into the wardrobe.
6. The task is completed.

By following this sequence, anyone can achieve the task. This idea is the basic principle behind sequence programming. What it effectively means is to break up a given task into an order or sequence of smaller tasks. Completing every small task or job in effect helps in completing the larger task.

Definition of a Sequence

A sequence in programming refers to an ordered set of instructions or tasks. In common parlance, one may also use the term “algorithm” which can be defined as an ordered sequence of steps to achieve a particular task. The order is key here.
If you follow the wrong order, then the task cannot be achieved, or you may get ambiguous results. Now that the definition of sequence is clear to you, let's go through another example of sequence.

What are Algorithms?

Algorithms are fundamental to sequences in programming, and they describe in detail how any task is to be achieved in a step-by-step ordered manner. Algorithms can also be said to be a series of steps taken to achieve a given task.
Computers rely heavily on algorithms to do useful work. Computers can neither function with intuitive knowledge, nor do they have their own intelligence to make decisions. They depend on humans to provide them with the “thought pattern” or the “method” of doing work.

Examples of Sequences

The project that has been assigned to you is to build a flower garden. The area of land is marked out for you, and all you have to do is start planting. Now you can approach the task haphazardly or systematically. Either way, you will get a result.

The reason to choose to do the work systematically is so that the task becomes more simplified. First, you can decide on a couple of things, for example, what you are going to plant, how you are going to arrange the plants, what compost and soil mixture you will use. Then you will begin planting from one end. You will divide your work into time slots and complete it in parts.

In effect first, you need to draw up a plan. Decide on the necessary steps to take and then achieve the task.

Let’s also assign this same task to a computer.

Computers cannot work haphazardly. Computers also lack intuition and decision-making capabilities. They need to be programmed to achieve the task. Now to program the computer means one has to give it a set of instructions. And if you want to achieve the task meaningfully, you need to give in the instructions in an ordered sequence. That way, the machine will be able to do the same task as you.

Here is a sequence you can give the computer to plant a garden.

1. Divide the land into four parts.
2. Plant flowers in the top-right part.
3. Plant vegetables in the top-left part.
4. Plant grass in the bottom right part.
5. Install a water fountain in the bottom left part.

This sequence makes the work meaningful as well as the computer can achieve it step-by-step. With the help of this example, what is sequence should be apparent to you?

Why do Computers Need Sequences?

Computers do not have a mind of their own. They lack intuitive thinking and expression of ideas. They are machines that are built and programmed to do certain tasks. They do not deviate from whatever they are made to do. In a way, they are the ideal minions, since they work without tiring and accurately follow instructions to the letter.

A computer system is a what-you-put-in-is-what-you-get. Program a computer to do irresponsible work, and that is what it will achieve. Program it to do intelligent work, and that is what it will achieve.

Computers need sequence programming because they lack the intelligence of their own, and they depend on sequenced instructions to complete work in record time. This manner of functioning makes computers both useful and versatile in a vast field of work.

What are the Benefits of Using Sequences in coding?

The first benefit of sequence programming is that it most closely mirrors the actual thought pattern. Our manner of thinking can be easily replicated in sequence programming. Humans think in an ordered manner when it comes to doing work so the same thought pattern can be lent to computers.

Also, sequence programming accurately reflects logical thinking. Computers should not work according to sensibilities but must use logic and reasoning to finish their tasks; hence this method of programming is very vital for them.

Thus, the importance of sequence thinking and why it is useful to a computer should be apparent to you. You can use this idea further to gain more expertise in computer science. It is both a vital and exciting concept.

MEMORY-LOCATIONS & ADDRESSES

Memory consists of many millions of storage cells (flip-flops). Each cell can store a bit of information i.e. 0 or 1 (Figure). Each group of n bits is referred to as a word of information, and n is called the word length. The word length can vary from 8 to 64 bits. A unit of 8 bits is called a byte.

Accessing the memory to store or retrieve a single item of information (word/byte) requires distinct addresses for each item location. (It is customary to use numbers from 0 through 2ˆk-1 as the addresses of successive-locations in the memory).

If 2ˆk = no. of addressable locations; then 2ˆk addresses constitute the address-space of the computer. For example, a 24-bit address generates an address-space of 2ˆ24 locations (16 mb)

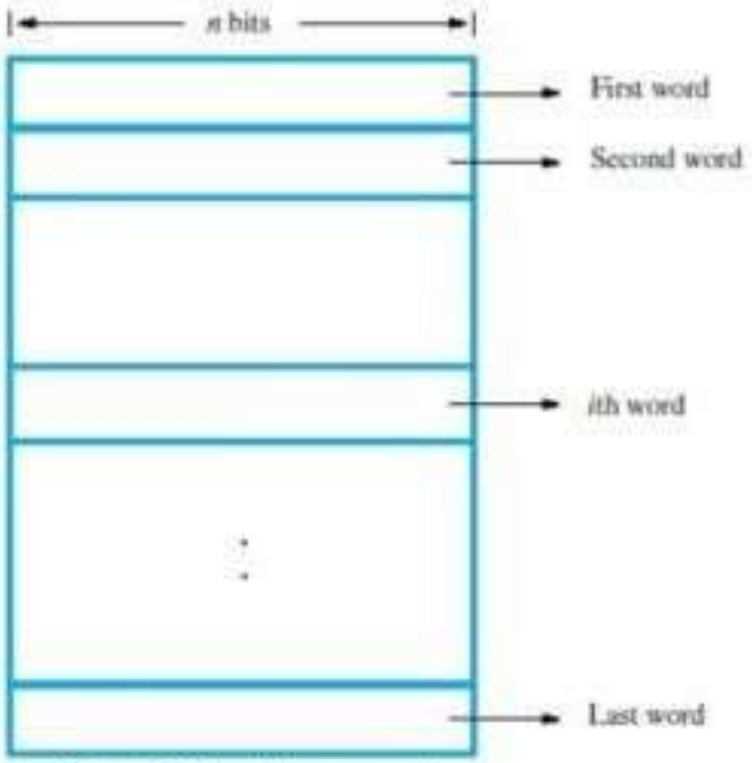


Figure 2.1 Memory words.

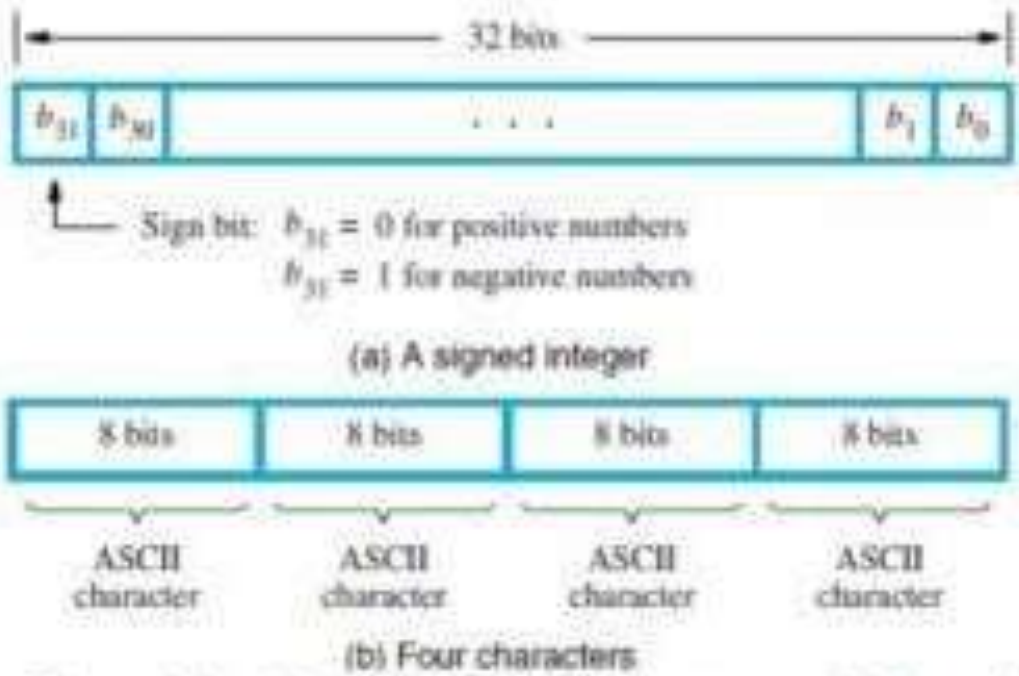


Figure 2.2 Examples of encoded information in a 32-bit word.

Instruction Format and Sequencing

Instructions in a program have proper instruction format and sequencing which decides the flow of the program execution. There are several types of instructions in a computer program like:

- nstruction performing arithmetic or the logic operation on the numbers
- instruction that branches to other instruction in the program
- the instruction that checks the particular condition and many more

Content: Instruction Format and Sequencing

1. Instruction Format &Types
2. Register Transfer Notation
3. Assembly Language Notation
4. RISC and CISC Instruction set
5. Instruction Sequencing and Execution
6. Branching Instruction

Instruction Format & Types

Before studying more about the instruction let us understand the basic format of instruction. Below is the figure showing the general format of the instruction:-

Instruction Format

The **Opcode** shown above determines the nature of the **operation** to be performed. The **operands**, Operand 1 and operand 2 identify the data on which operation has to be performed. The operand can be a **memory location, a processor register, immediate value** or **logical data.**

Now, as we have studied above, the instructions could be of many types. The operation specified in the instruction decides the nature of that instruction. Generalizing all types of instructions, they can be **categorized** into four types as follows:

1. Instructions that **transfers** the data between the memory of the computer and processor register.
2. Instructions that can perform **arithmetic and logical operations** on the data.
3. Instructions controlling the **sequencing** of execution of instructions, like branch instruction.
4. Instructions performing the **I/O transfer** i.e. transferring the data from internal storage to the external storage or device.

Register Transfer Notation

When it comes to 'information transfer' it requires a **source** location and a **destination** location. The source location is from where the value is to be read and the destination location is where the value is to be stored.

Here, the source or destination locations can be **memory locations**, **processor registers** or they can be the **register in I/O subsystem**. These locations are given a symbolic name for the convenience.

Below are the instructions for register transfer notations:

Instruction 1 R4 <- [VAR3]

Instruction 2 R4 <- [R3] + [R2]

The instruction 1 indicates the value in the memory location having symbolic name V2AR3 is transferred to the processor register R4. Here, VAR 3 is placed in square bra

cket [ ], as the square bracket around a location denotes the content or value stored at that location.

The old content of R4 will get overwritten by the value of VAR3.

Similarly, in instruction 2 the value inside the processor register R3 and R2 is added and then stored in the processor register R4. Here, also the old content of R4 +will get overwritten by the sum of R3 and R2.

These kinds of notations are called as **Register Transfer Notation (RTN).**

Always remember the value at the right-hand side of the RTN expression is value to be operated or considered for the transfer and the left-hand side is the name of the location where that value has to be stored.

The contents of operands mentioned at the RHS of RTN expression does not changes. But, the contents of the operand mention at LHS of RTN expression gets overwritten.

Assembly Language Notation

Assembly Language notations are used to represent the **machine instructions**. An assembly language instruction specifies the **operation**, that has to be performed and the **operands** that will be involved in the operation.

Assembly language instruction use **mnemonics** to denote the operations. These mnemonics are nothing but the abbreviation of the word that describes the nature of operation. Like, for addition it has Add, for Multiplication it has Mul, for division Div.

Let us overview the assembly language instruction.

Instruction 1 Load R4, VAR3

Instruction 2 Add R4, R3, R2

In instruction 1 the content in memory location VAR3 is loaded in processor register R4. After the execution of this instruction, the content of VAR3 will remain the same as before the execution of the instruction. But, the old content of R4 is overwritten by the content of VAR3.

In instruction 2 the content of the registers R3 and R2 are added and the sum is stored in register R4.

RISC and CISC Instruction set

Computers can be differentiated on the basis of the nature of their instruction set. There are two ways in which the instruction set for modern-day computer could be designed those are RISC and CISC.

Talking about ***Complex Instruction Set Computers (CISC)***, they were prevalent in the 1970s before the Reduced Instruction Set Computers (RISC) were introduced.

CSIC has **complicated** instruction sets which specifies more **complicated** operations. Some instructions in CISC would occupy **more than one word** in memory. Most of the modern-day CISC instruction has **two address format**.

Suppose we have to execute the below instruction in CISC instruction format.

C = A + B

In CISC format it would be as follow:

Move C, B

Add C, A

The content of memory location A and B will not get changed whereas, the content in C would be overwritten.

Move instruction has the general instruction format:

Move *destination, source*

Here, the **source and destination** could either be a **memory location or a processor register**.

The **Move** instruction has functionality similar to Load and Store instruction of RISC. But it has a difference too.

Like, in Load instruction source is a memory location and destination is a register. While in Store instruction source is a register and destination is a memory location.

But, in Move instruction both source and destination can either be a memory location or processor register or it can also be used to move **immediate** operands.

Now, ***Reduced Instruction Set Computers (RISC)*** has two important characteristics those are:

1. Each instruction should in a **single word** in memory.
2. It has **Load/Store** instruction to **access** memory **operands**.

In RISC instruction performing arithmetic and logic operation, *all the operands must be in the processor register or either of one operand must be explicitly mentioned within the instruction*. This is because RISC instruction can perform the arithmetic and logic operation *only on the operand of processor registers*.

At the time of program execution, all the instructions and data of the program must be present in computer memory. If some operands are expected to be in processor register before the operands can be operated, then it is done using Load instruction.

If we have to produce RISC instruction set for the following instruction:

C = A+ B

In RISC format the above instruction is:

Load R2, A

Load R3, B

Add R4, R3, R2 // *three address format*

Store R4, C

The general format for Load and Store:

Load destination, source //*destination is processor register & source is a memory location*

Store source, destination // *destination is a memory location & source is a processor register*

Instruction Sequencing and Execution

Execution of instruction is a two-step procedure. In the first step, the instruction is **fetched** from memory. In the second step, the instruction is executed.

First Step-> Instruction Fetch

The processor has a register named **program counter** which has the address of the instruction that has to be executed next. To start the execution of a program, the address of first instruction has to be placed in the program counter.

After the address of first instruction is placed in the program counter, the processor uses this address to fetch the first instruction. As soon as the first instruction is fetched the content in program counter is incremented by the word length (the group of n bits that can store a single, basic instruction) i.e. now, it has the address of next successive instruction to be executed.

After fetching the instruction from memory location it is placed in the **instruction register** of the processor.

Second Step -> Instruction execution

In the second step, the processor examines the instruction present in the **instruction register** and find out which operation has to be performed.

It first fetches the **operands** involved in the instruction from the memory or processor register. Then the processor performs the **arithmetic and logical operation** and **stores** the result in the destination location mention in the instruction.

During this time the content of program counter is incremented so that it could point next instruction for execution. This execution of instructions in increasing order of addresses is **straight line sequencing**.

Branching Instruction

Now, suppose we have to add n numbers. Their addresses in the symbolic form are Num1, Num2, Num3,…….NumN. So, the instructions for adding these n numbers are as follow:

Load R2, Num1

Load R3, Num2

Add R2, R3, R2

Load R3, Num3

Add R2, R3, R2

Load R3, Num4

Add R2, R3, R2

.

.

.

.

Load R3, NumN

Add R2, R3, R2

Store R2 SUM

Instead of writing this straight-line sequence of instruction we could implement a **loop** to this program. To add n numbers a loop has to be repeated n number of times.

Suppose there are n numbers to be added. So, this count value n is stored in the memory location N. Below is the instruction set specifying the loop to add n numbers

Load R2, N

Clear R3

Determine Next_number // *Loop start*

Load R5, Next_number

Add R3, R5

Sub R2, R2, #1

Branch if [R2]>0 Loop // *Loop repeat*

Store R3, SUM

In the above code “Branch if [R2]>0 Loop” is a **branch instruction**. This instruction loads the address of the next number in the program counter. The address of the next number to be added is a **branch target**.

The branch instruction we have used is the **conditional branch** instruction as it will perform loop until the value of R2 >0. As the condition R2>0 will become false it will stop performing the loop.

Multiplication Algorithm in Signed Magnitude Representation

- Difficulty Level : Medium
- Last Updated : 21 Aug, 2019

Multiplication of two fixed point binary number in *signed magnitude representation* is done with process of *successive shift* and *add operation*.

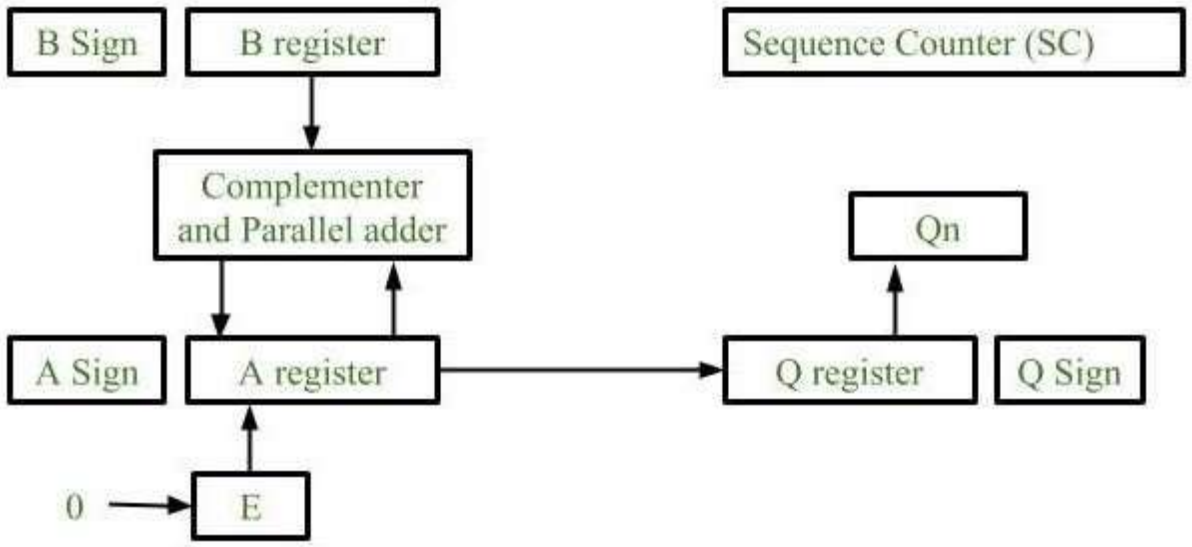
```
   10111 (Multiplicand)
x  10011 (Multiplier)
  -------
   10111
   10111
  00000
 00000
10111
---------
011011010     (Product)
```

In the multiplication process we are considering successive bits of the multiplier, least significant bit first.
If the multiplier bit is 1, the multiplicand is copied down else 0’s are copied down.

The numbers copied down in successive lines are shifted one position to the left from the previous number.
Finally numbers are added and their sum form the product.

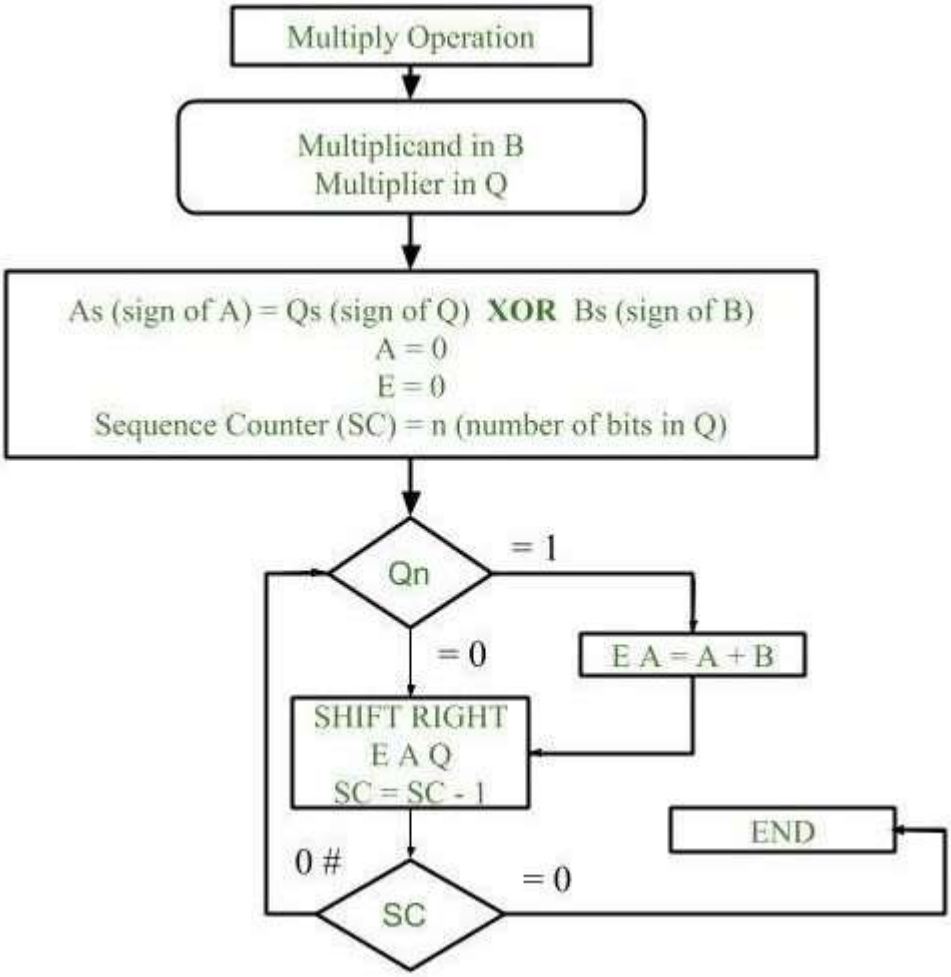
The sign of the product is determined from the sign of the multiplicand and multiplier. If they are alike, sign of the product is positive else negative.

Hardware Implementation :
Following components are required for the *Hardware Implementation* of multiplication algorithm :



1. **Registers:**
   Two Registers B and Q are used to store multiplicand and multiplier respectively. Register A is used to store partial product during multiplication. Sequence Counter register (SC) is used to store number of bits in the multiplier.
2. **Flip Flop:**
   To store sign bit of registers we require three flip flops (A sign, B sign and Q sign). Flip flop E is used to store carry bit generated during partial product addition.
3. **Complement and Parallel adder:**
   This hardware unit is used in calculating partial product i.e, perform addition required.

Flowchart of Multiplication:



1. Initially multiplicand is stored in B register and multiplier is stored in Q register.
2. Sign of registers B (Bs) and Q (Qs) are compared using **XOR** functionality (i.e., if both the signs are alike, output of XOR operation is 0 unless 1) and output stored in As (sign of A register).
   Note: Initially 0 is assigned to register A and E flip flop. Sequence counter is initialized with value n, n is the number of bits in the Multiplier.
3. Now least significant bit of multiplier is checked. If it is 1 add the content of register A with Multiplicand (register B) and result is assigned in A register with carry bit in flip flop E. Content of E A Q is shifted to right by one position, i.e., content of E is shifted to most significant bit (MSB) of A and least significant bit of A is shifted to most significant bit of Q.
4. If Qn = 0, only shift right operation on content of E A Q is performed in a similar fashion.
5. Content of Sequence counter is decremented by 1.
6. Check the content of Sequence counter (SC), if it is 0, end the process and the final product is present in register A and Q, else repeat the process.

Example:

Multiplicand = 10111

Multiplier = 10011

Multiplicand B = 10111	E	A	Q	SC
Multiplier in Q	0	00000	10011	101
Qn = 1; add B		10111		
First partial product	0	10111		
Shift right EAQ	0	01011	11001	100
Qn = 1; add B		10111		
Second partial product	1	00010		
Shift right EAQ	0	10001	01100	011
Qn = 0; shift right EAQ	0	01000	10110	010
Qn = 0; shift right EAQ	0	00100	01011	001
Qn = 1; add B		10111		
Fifth partial product	0	11011		
Shift right EAQ	0	01101	10101	000

Final product in AQ
0110110101

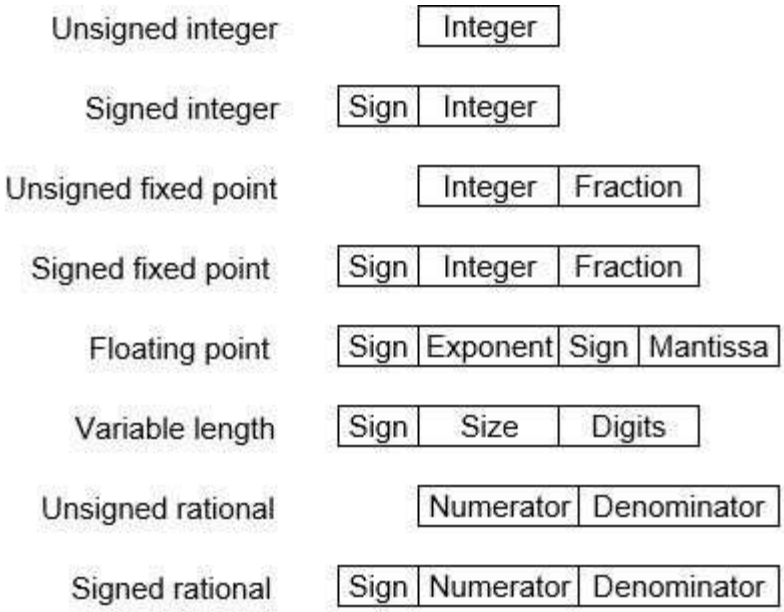
Fixed Point and Floating Point Number Representations

Digital Computers use Binary number system to represent all types of information inside the computers. Alphanumeric characters are represented using binary bits (i.e., 0 and 1). Digital representations are easier to design, storage is easy, accuracy and precision are greater.

There are various types of number representation techniques for digital number representation, for example: Binary number system, octal number system, decimal number system, and hexadecimal number system etc. But Binary number system is most relevant and popular for representing numbers in digital computer system.

Storing Real Number

These are structures as following below −



There are two major approaches to store real numbers (i.e., numbers with fractional component) in modern computing. These are (i) Fixed Point Notation and (ii) Floating Point Notation. In fixed point notation, there are a fixed number of digits after the decimal point, whereas floating point number allows for a varying number of digits after the decimal point.

Fixed-Point Representation −

This representation has fixed number of bits for integer part and for fractional part. For example, if given fixed-point representation is IIII.FFFF, then you can store minimum value is 0000.0001 and maximum value is 9999.9999. There are three parts of a fixed-point number representation: the sign field, integer field, and fractional field.

We can represent these numbers using:

- Signed representation: range from $-(2^{(k-1)}-1)$ to $(2^{(k-1)}-1)$, for k bits.
- 1's complement representation: range from $-(2^{(k-1)}-1)$ to $(2^{(k-1)}-1)$, for k bits.
- 2's complementation representation: range from $-(2^{(k-1)})$ to $(2^{(k-1)}-1)$, for k bits.

2's complementation representation is preferred in computer system because of unambiguous property and easier for arithmetic operations.

Example −Assume number is using 32-bit format which reserve 1 bit for the sign, 15 bits for the integer part and 16 bits for the fractional part.

Then, -43.625 is represented as following:

1	000000000101011	1010000000000000
Sign bit	Integer part	Fractional part

Where, 0 is used to represent + and 1 is used to represent. 000000000101011 is 15 bit binary value for decimal 43 and 1010000000000000 is 16 bit binary value for fractional 0.625.

The advantage of using a fixed-point representation is performance and disadvantage is relatively limited range of values that they can represent. So, it is usually inadequate for numerical analysis as it does not allow enough numbers and accuracy. A number whose representation exceeds 32 bits would have to be stored inexactly.

Smallest	0	000000000000000	0000000000000001
	Sign bit	Integer part	Fractional part
Largest	0	111111111111111	1111111111111111
	Sign bit	Integer part	Fractional part

These are above smallest positive number and largest positive number which can be store in 32-bit representation as given above format. Therefore, the smallest positive number is $2^{-16} \approx 0.000015$ approximate and the largest positive number is $(2^{15}-1)+(1-2^{-16})=2^{15}(1-2^{-16}) =32768$, and gap between these numbers is 2^{-16}.

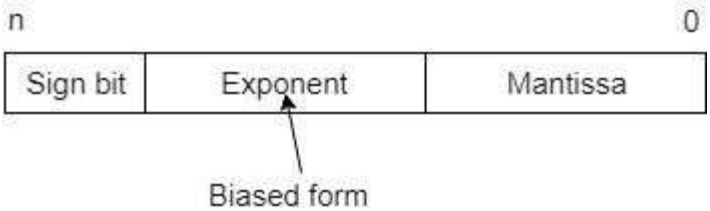
We can move the radix point either left or right with the help of only integer field is 1.

Floating-Point Representation −

This representation does not reserve a specific number of bits for the integer part or the fractional part. Instead it reserves a certain number of bits for the number (called the mantissa or significand) and a certain number of bits to say where within that number the decimal place sits (called the exponent).

The floating number representation of a number has two part: the first part represents a signed fixed point number called mantissa. The second part of designates the position of the decimal (or binary) point and is called the exponent. The fixed point mantissa may be fraction or an integer. Floating -point is always interpreted to represent a number in the following form: Mxr^e.

Only the mantissa m and the exponent e are physically represented in the register (including their sign). A floating-point binary number is represented in a similar manner except that is uses base 2 for the exponent. A floating-point number is said to be normalized if the most significant digit of the mantissa is 1.



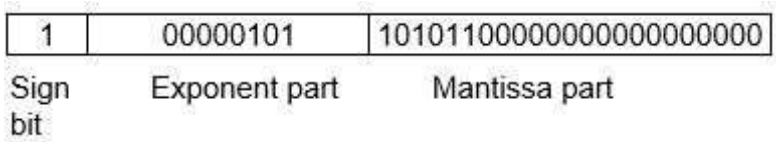
So, actual number is $(-1)^s(1+m)x2^{(e-Bias)}$, where *s* is the sign bit, *m* is the mantissa, *e* is the exponent value, and *Bias* is the bias number.

Note that signed integers and exponent are represented by either sign representation, or one's complement representation, or two's complement representation.

The floating point representation is more flexible. Any non-zero number can be represented in the normalized form of $\pm(1.b_1b_2b_3 \ldots)_2x2^n$ This is normalized form of a number x.

Example −Suppose number is using 32-bit format: the 1 bit sign bit, 8 bits for signed exponent, and 23 bits for the fractional part. The leading bit 1 is not stored (as it is always 1 for a normalized number) and is referred to as a *"hidden bit*".

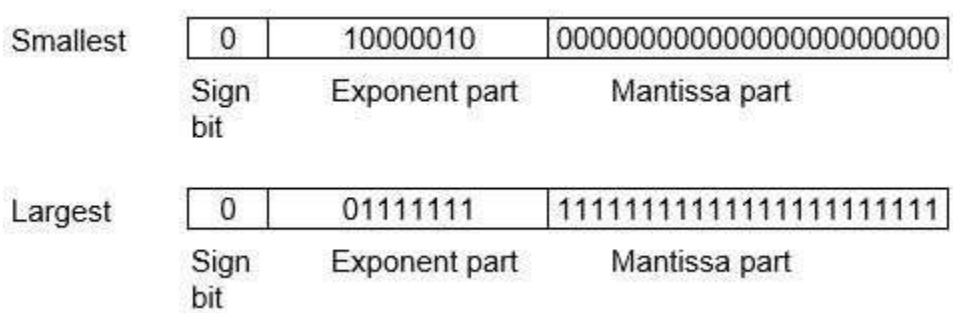
Then −53.5 is normalized as $-53.5=(-110101.1)_2=(-1.101011)x2^5$, which is represented as following below,



Where 00000101 is the 8-bit binary value of exponent value +5.

Note that 8-bit exponent field is used to store integer exponents $-126 \leq n \leq 127$.

The smallest normalized positive number that fits into 32 bits is $(1.00000000000000000000000)_2x2^{-126}=2^{-126}\approx 1.18x10^{-38}$, and largest normalized positive number that fits into 32 bits is $(1.11111111111111111111111)_2x2^{127}=(2^{24}-1)x2^{104} \approx 3.40x10^{38}$. These numbers are represented as following below,



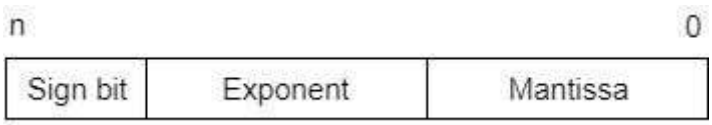
The precision of a floating-point format is the number of positions reserved for binary digits plus one (for the hidden bit). In the examples considered here the precision is 23+1=24.

The gap between 1 and the next normalized floating-point number is known as machine epsilon. the gap is $(1+2^{-23})-1=2^{-23}$for above example, but this is same as the smallest positive floating-point number because of non-uniform spacing unlike in the fixed-point scenario.

Note that non-terminating binary numbers can be represented in floating point representation, e.g., 1/3 = $(0.010101 \ldots)_2$ cannot be a floating-point number as its binary representation is non-terminating.

IEEE Floating point Number Representation −

IEEE (Institute of Electrical and Electronics Engineers) has standardized Floating-Point Representation as following diagram.



So, actual number is $(-1)^s(1+m)x2^{(e-Bias)}$, where *s* is the sign bit, *m* is the mantissa, *e* is the exponent value, and *Bias* is the bias number. The sign bit is 0 for positive number and 1 for negative number. Exponents are represented by or two's complement representation.

According to IEEE 754 standard, the floating-point number is represented in following ways:

- Half Precision (16 bit): 1 sign bit, 5 bit exponent, and 10 bit mantissa
- Single Precision (32 bit): 1 sign bit, 8 bit exponent, and 23 bit mantissa
- Double Precision (64 bit): 1 sign bit, 11 bit exponent, and 52 bit mantissa
- Quadruple Precision (128 bit): 1 sign bit, 15 bit exponent, and 112 bit mantissa

Special Value Representation −

There are some special values depended upon different values of the exponent and mantissa in the IEEE 754 standard.

- All the exponent bits 0 with all mantissa bits 0 represents 0. If sign bit is 0, then +0, else -0.
- All the exponent bits 1 with all mantissa bits 0 represents infinity. If sign bit is 0, then +∞, else -∞.
- All the exponent bits 0 and mantissa bits non-zero represents denormalized number.
- All the exponent bits 1 and mantissa bits non-zero represents error.

What is Floating-Point Representation in Computer Architecture?

The floating-point representation can implement operations for high range values. The numerical evaluations are carried out using floating-point values. It can create calculations easy, scientific numbers are described as follows −

The number 5,600,000 can be described as 0.56 * 107.

Therefore, 0.56 is the mantissa and 7 is the value of the exponent.

Binary numbers can also be described in exponential form. The description of binary numbers in the exponential form is called floating-point representation. The floating-point representation breaks the number into two parts, the left-hand side is a signed, fixed-point number known as a mantissa and the right-hand side of the number is known as the exponent. The floating-point values are also authorized with a sign; 0 denoting the positive value and 1 denoting the negative value.

The general structure of floating-point representation of a binary number −

$x=(x_0*2^0+x_1*2^1+x_2*2^2\pm--\mp b_{-(n-1)}*2^{-(n-1)})$

$\text{mantissa}*2^{\text{Exponent}}$

In the following syntax, the decimal point is transferred left for negative exponents of two and right for positive exponents of two. Both the mantissa and the exponent are signed values enabling negative numbers and negative exponents commonly.

Example − Convert 111101.1000110 into floating-point value.

111101.1000110 = 1.111011000110 * 2^5Converted to floating-point value

→ Denotes negative sign value

In this example, the integer value is converted to a floating-point value by changing the radix point next to the signed integer and scaling up the number to the exponential form by multiplying the value with the base 2. The value remains unaltered and this phase is known as the normalized method.

Computer Architecture: Input/Output Organisation

Input/Output Subsystem

The I/O subsystem of a computer provides an efficient mode of communication between the central system and the outside environment. It handles all the input-output operations of the computer system.

Peripheral Devices

Input or output devices that are connected to computer are called **peripheral devices**. These devices are designed to read information into or out of the memory unit upon command from the CPU and are considered to be the part of computer system. These devices are also called **peripherals**.

For example: *Keyboards*, *display units* and *printers* are common peripheral devices.

There are three types of peripherals:

1. **Input peripherals** : Allows user input, from the outside world to the computer. Example: Keyboard, Mouse etc.

2. **Output peripherals**: Allows information output, from the computer to the outside world. Example: Printer, Monitor etc

3. **Input-Output peripherals**: Allows both input(from outised world to computer) as well as, output(from computer to the outside world). Example: Touch screen etc.

Interfaces

Interface is a shared boundary btween two separate components of the computer system which can be used to attach two or more components to the system for communication purposes.

There are two types of interface:

1. CPU Inteface

2. I/O Interface

Let's understand the I/O Interface in details,

Input-Output Interface

Peripherals connected to a computer need special communication links for interfacing with CPU. In computer system, there are special hardware components between the CPU and peripherals to control or manage the input-output transfers. These components are called **input-output interface units** because they provide communication links between processor bus and peripherals. They provide a method for transferring information between internal system and input-output devices.

Modes of I/O Data Transfer

Data transfer between the central unit and I/O devices can be handled in generally three types of modes which are given below:

1. Programmed I/O

2. Interrupt Initiated I/O

3. Direct Memory Access

Programmed I/O

Programmed I/O instructions are the result of I/O instructions written in computer program. Each data item transfer is initiated by the instruction in the program.

Usually the program controls data transfer to and from CPU and peripheral. Transferring data under programmed I/O requires constant monitoring of the peripherals by the CPU.

Interrupt Initiated I/O

In the programmed I/O method the CPU stays in the program loop until the I/O unit indicates that it is ready for data transfer. This is time consuming process because it keeps the processor busy needlessly.

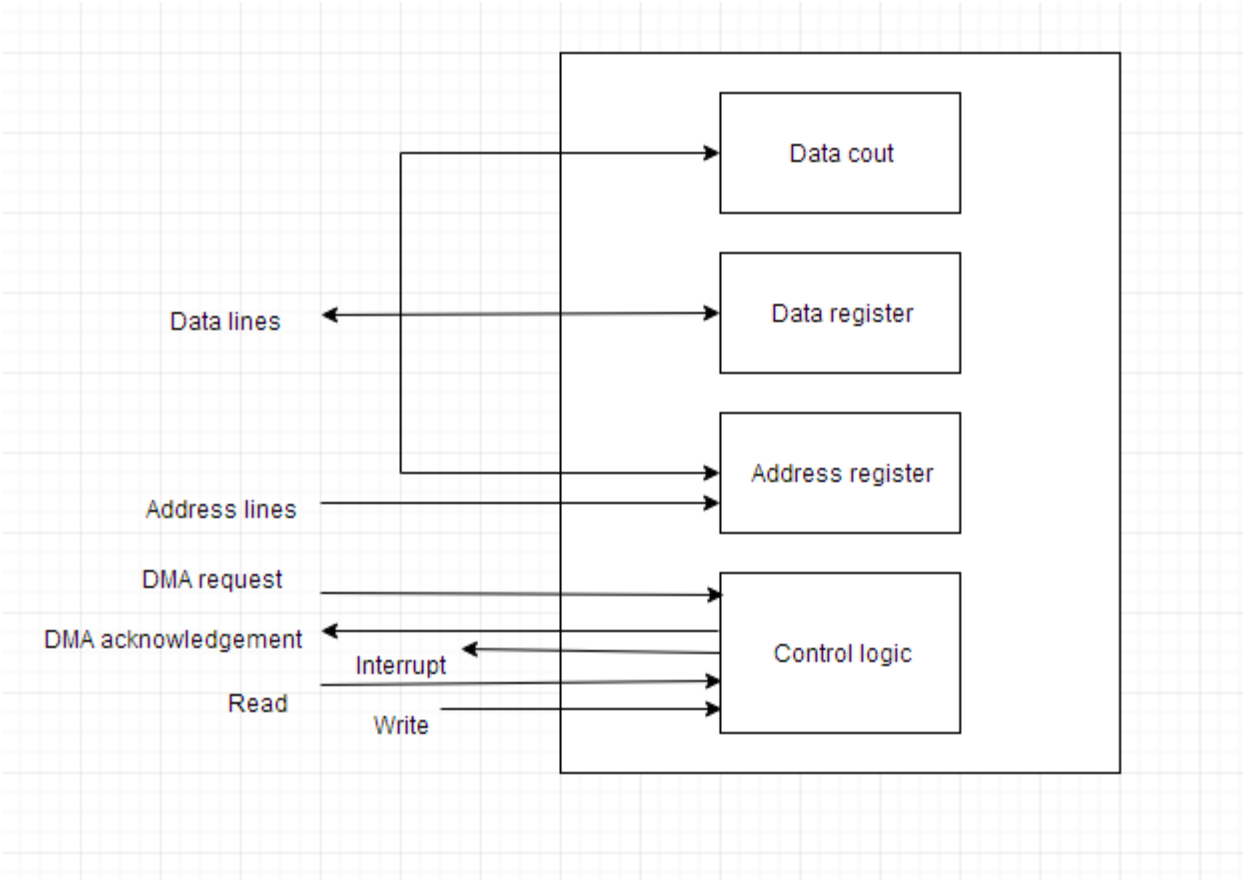
This problem can be overcome by using **interrupt initiated I/O**. In this when the interface determines that the peripheral is ready for data transfer, it generates an interrupt. After receiving the interrupt signal, the CPU stops the task which it is processing and service the I/O transfer and then returns back to its previous processing task.

Direct Memory Access

Removing the CPU from the path and letting the peripheral device manage the memory buses directly would improve the speed of transfer. This technique is known as **DMA**.

In this, the interface transfer data to and from the memory through memory bus. A DMA controller manages to transfer data between peripherals and memory unit.

Many hardware systems use DMA such as disk drive controllers, graphic cards, network cards and sound cards etc. It is also used for intra chip data transfer in multicore processors. In DMA, CPU would initiate the transfer, do other operations while the transfer is in progress and receive an interrupt from the DMA controller when the transfer has been completed.



Above figure shows block diagram of DMA

An input-output processor (IOP) is a processor with direct memory access capability. In this, the computer system is divided into a memory unit and number of processors.

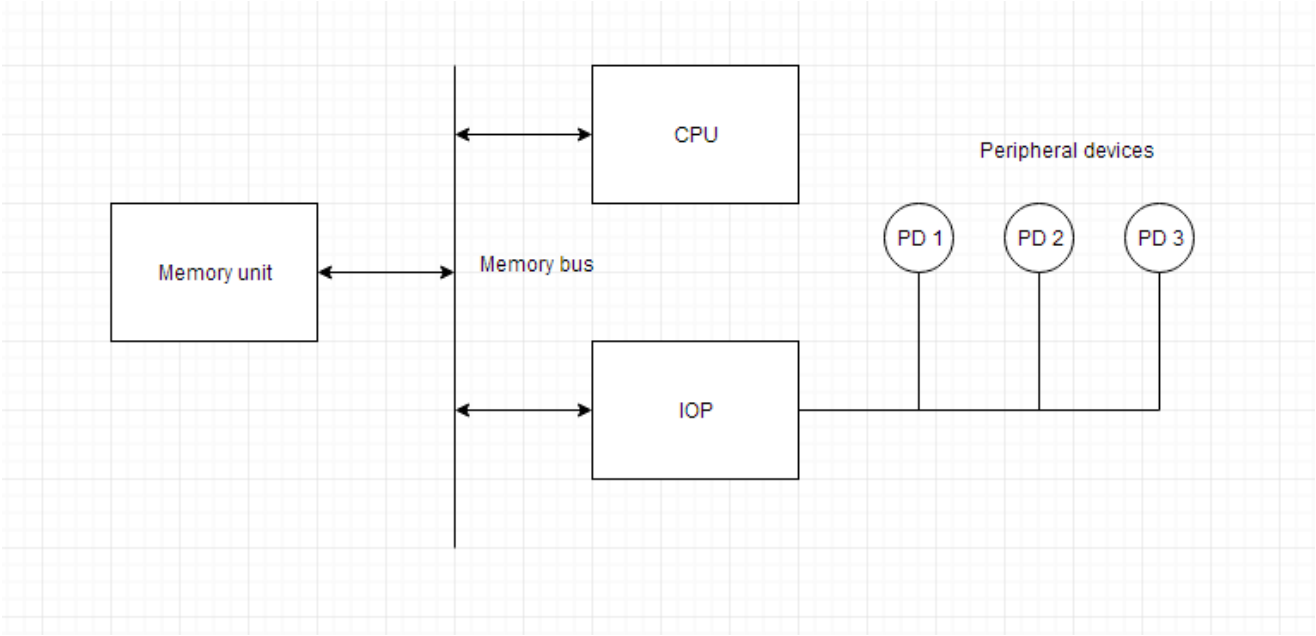
Each IOP controls and manage the input-output tasks. The IOP is similar to CPU except that it handles only the details of I/O processing. The IOP can fetch and execute its own instructions. These IOP instructions are designed to manage I/O transfers only.

Block Diagram Of I/O Processor

Below is a block diagram of a computer along with various I/O Processors. The memory unit occupies the central position and can communicate with each processor.

The CPU processes the data required for solving the computational tasks. The IOP provides a path for transfer of data between peripherals and memory. The CPU assigns the task of initiating the I/O program.

The IOP operates independent from CPU and transfer data between peripherals and memory.



The communication between the IOP and the devices is similar to the program control method of transfer. And the communication with the memory is similar to the direct memory access method.

In large scale computers, each processor is independent of other processors and any processor can initiate the operation.

The CPU can act as master and the IOP act as slave processor. The CPU assigns the task of initiating operations but it is the IOP, who executes the instructions, and not the CPU. CPU instructions provide operations to start an I/O transfer. The IOP asks for CPU through interrupt.

Instructions that are read from memory by an IOP are also called *commands* to distinguish them from instructions that are read by CPU. Commands are prepared by programmers and are stored in memory. Command words make the program for IOP. CPU informs the IOP where to find the commands in memory.

Introduction of Stack based CPU Organization

The computers which use Stack-based CPU Organization are based on a data structure called a **stack**. The stack is a list of data words. It uses the **Last In First Out (LIFO)** access method which is the most popular access method in most of the CPU. A register is used to store the address of the topmost element of the stack which is known as **Stack pointer (SP)**. In this organization, ALU operations are performed on stack data. It means both the operands are always required on the stack. After manipulation, the result is placed in the stack.
The main two operations that are performed on the operators of the stack are **Push** and **Pop**. These two operations are performed from one end only.

1. Push –

This operation results in inserting one operand at the top of the stack and it decreases the stack pointer register. The format of the PUSH instruction is:

PUSH

It inserts the data word at a specified address to the top of the stack. It can be implemented as:

//decrement SP by 1

SP <-- SP - 1

//store the content of specified memory address

//into SP; i.e, at top of stack

SP <-- (memory address)

2. Pop –
This operation results in deleting one operand from the top of the stack and increasing the stack pointer register. The format of the POP instruction is:
POP

It deletes the data word at the top of the stack to the specified address. It can be implemented as:

//transfer the content of SP (i.e, at top most data)

//into specified memory location

(memory address) <-- SP

//increment SP by 1

SP <-- SP + 1

Operation type instruction does not need the address field in this CPU organization. This is because the operation is performed on the two operands that are on the top of the stack. For example:

SUB

This instruction contains the opcode only with no address field. It pops the two top data from the stack, subtracting the data, and pushing the result into the stack at the top.

PDP-11, Intel's 8085, and HP 3000 are some examples of stack-organized computers.

The advantages of Stack-based CPU organization –
- Efficient computation of complex arithmetic expressions.
- Execution of instructions is fast because operand data are stored in consecutive memory locations.
- The length of instruction is short as they do not have an address field.

The disadvantages of Stack-based CPU organization –
- The size of the program increases.

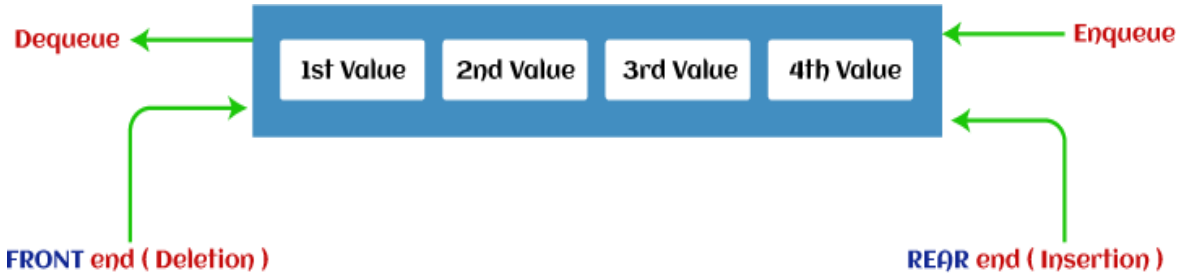
Note: Stack-based *CPU* organization *uses zero address instruction.*

What is a Queue?

Queue is the data structure that is similar to the queue in the real world. A queue is a data structure in which whatever comes first will go out first, and it follows the FIFO (First-In-First-Out) policy. Queue can also be defined as the list or collection in which the insertion is done from one end known as the **rear end** or the **tail** of the queue, whereas the deletion is done from another end known as the **front end** or the **head** of the queue.

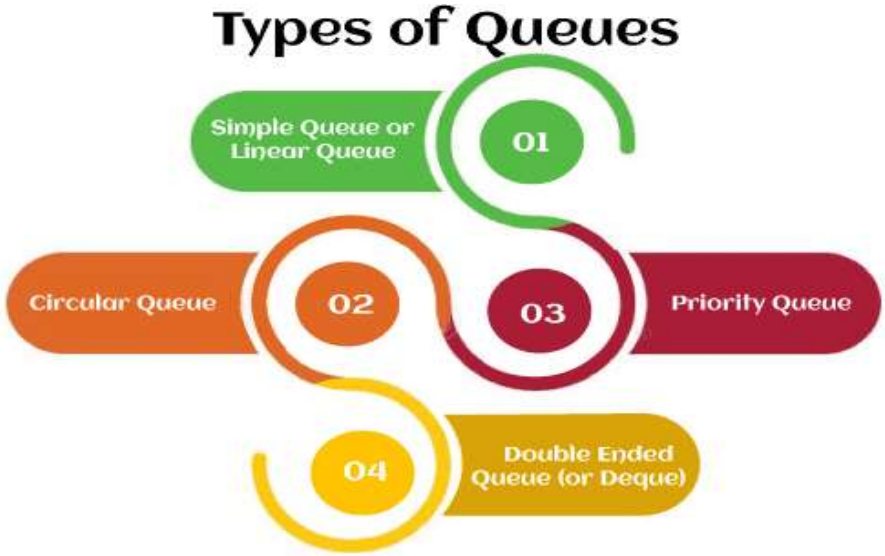
The real-world example of a queue is the ticket queue outside a cinema hall, where the person who enters first in the queue gets the ticket first, and the last person enters in the queue gets the ticket at last. Similar approach is followed in the queue in data structure.

The representation of the queue is shown in the below image **tay**



Types of Queue

There are four different types of queue that are listed as follows -



- Simple Queue or Linear Queue
- Circular Queue

- Priority Queue
- Double Ended Queue (or Deque)

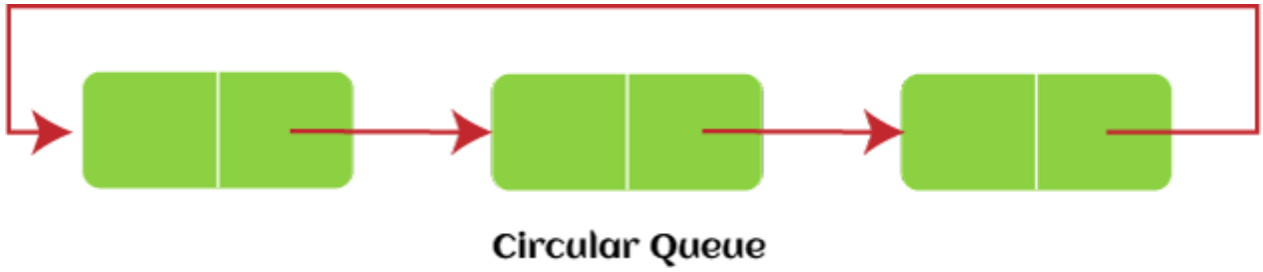
Simple Queue or Linear Queue

In Linear Queue, an insertion takes place from one end while the deletion occurs from another end. The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule.

The major drawback of using a linear Queue is that insertion is done only from the rear end. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue. In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.

Circular Queue

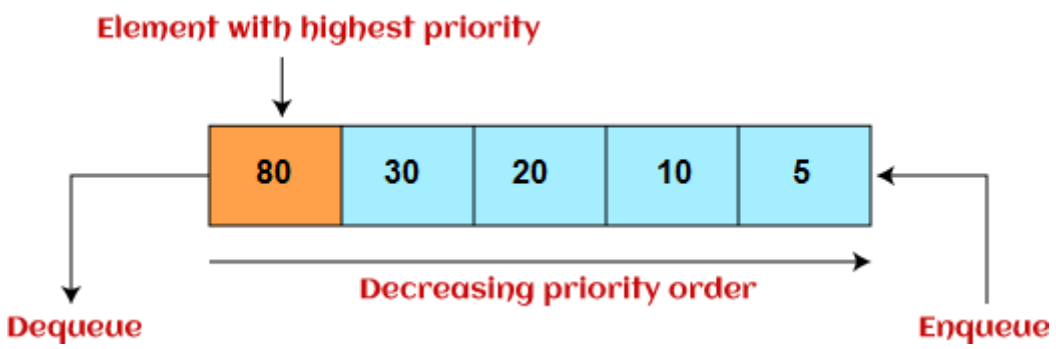
In Circular Queue, all the nodes are represented as circular. It is similar to the linear Queue except that the last element of the queue is connected to the first element. It is also known as Ring Buffer, as all the ends are connected to another end. The representation of circular queue is shown in the below image -



The drawback that occurs in a linear queue is overcome by using the circular queue. If the empty space is available in a circular queue, the new element can be added in an empty space by simply incrementing the value of rear. The main advantage of using the circular queue is better memory utilization.

Priority Queue

It is a special type of queue in which the elements are arranged based on the priority. It is a special type of queue data structure in which every element has a priority associated with it. Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle. The representation of priority queue is shown in the below image -



Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority. Priority queue is mainly used to implement the CPU scheduling algorithms.

There are two types of priority queue that are discussed as follows -

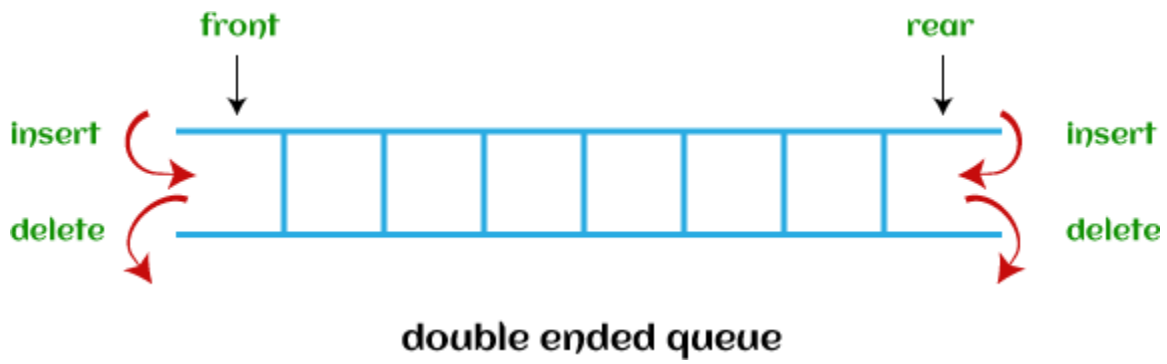
- **Ascending priority queue -** In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.
- **Descending priority queue -** In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

Deque (or, Double Ended Queue)

In Deque or Double Ended Queue, insertion and deletion can be done from both ends of the queue either from the front or rear. It means that we can insert and delete elements from both front and rear ends of the queue. Deque can be used as a palindrome checker means that if we read the string from both ends, then the string would be the same.

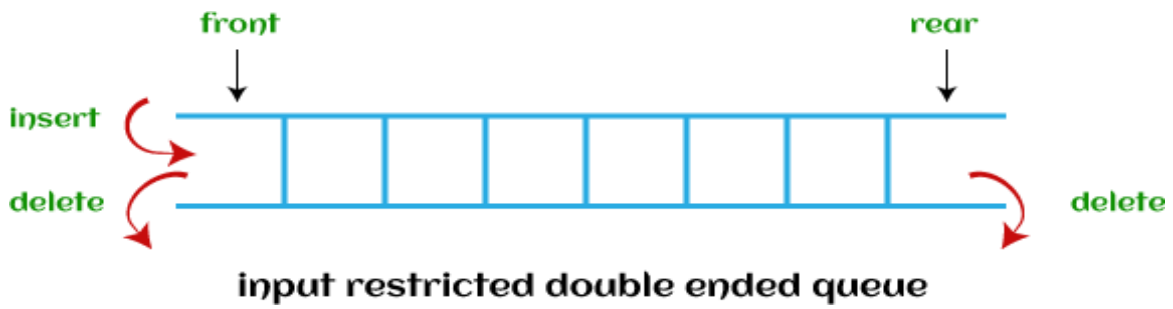
Deque can be used both as stack and queue as it allows the insertion and deletion operations on both ends. Deque can be considered as stack because stack follows the LIFO (Last In First Out) principle in which insertion and deletion both can be performed only from one end. And in deque, it is possible to perform both insertion and deletion from one end, and Deque does not follow the FIFO principle.

The representation of the deque is shown in the below image -



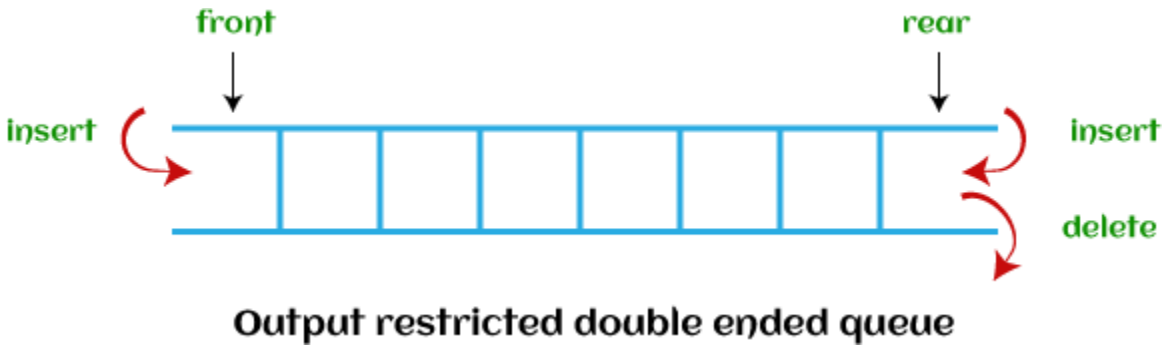
There are two types of deque that are discussed as follows -

- **Input restricted deque -** As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



- **Output restricted deque -** As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both

ends.



- **Queue overflow (isfull):** It shows the overflow condition when the queue is completely full.
- **Queue underflow (isempty):** It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

SUBROUTINES IN COMPUTER ORGANIZATION

Subroutines are programs that are used by other routines to accomplish a particular task. A subroutine can be called from any point within the main body of the micro-program. Frequently, many micro-programs contain identical sections of code. Microinstructions can be saved by employing subroutines that use common sections of microcode.

For example, the sequence of micro-operations needed to generate the effective address of the operand for instruction is common to all memory reference instructions. This sequence could be a subroutine that is called from within many other routines to execute the effective address computation.

Micro-programs that use subroutines must have a provision for storing the return address during a subroutine call and restoring the address during a subroutine return. This may be accomplished by placing the incremented output from the control address register into a subroutine register and branching to the beginning of the subroutine.

The subroutine register can then become the source for transferring the address for the return to the main routine. The best way to structure a register file that stores addresses for subroutines is to organize the registers in a last-in, first-out (LIFO) stack.

The instruction that transfers program control to a subroutine is known by different names. The most common names used are called subroutine, jump to the subroutine, branch to the subroutine, or branch and save the address.

A call subroutine instruction consists of an operation code together with an address that specifies the beginning of the subroutine.

The instruction is executed by performing two operations are as follows −

- The address of the next instruction available in the program counter (the return address) is stored in a temporary location so the subroutine knows where to return.
- The control is transferred to the beginning of the subroutine. The last instruction of every subroutine commonly called return from a subroutine transfers the return address from the temporary location into the program counter. This results in a transfer of program control to the instruction whose address was originally stored in the temporary location.

A subroutine call is implemented with the following micro-operations −

SP ← SP − 1	It can decrement the stack pointer.
M[SP] ← PC	It is used to push the content of the PC onto the stack.
PC ← Effective Address	It can transfer control to the subroutine.

If another subroutine is called by the current subroutine, the new return address is pushed into the stack, and so on. The instruction that returns from the last subroutine is implemented by the micro-operations −

PC ← M[SP]	It is used to pop the stack and transfer it to PC.
SP ← SP + 1	It can increment the stack pointer.

By using a subroutine stack, all return addresses are automatically stored by the hardware in one unit. The programmer does not have to be concerned or remember where the return address was stored.

A recursive subroutine is a subroutine that calls itself. If only one register or memory location can save the return address, and the recursive subroutine calls itself, it ends the previous return address.

This is undesirable because vital information is destroyed. This problem can be solved if different storage locations are employed for each use of the subroutine while another lighter-level use is still active.

When a stack is used, each return address can be pushed into the stack without destroying any previous values. This solves the problem of recursive subroutines because the next subroutine to exit is always the last subroutine that was called.

End